



Deployment Guide - Trader Nightmare

Comprehensive guide for deploying the Trader Nightmare cryptocurrency trading bot to production environments.



Quick Deployment Summary

Recommended Platform: Render (Best Docker support, transparent pricing, production-ready)

Alternative Options:

- Railway (Good for prototyping, usage-based pricing)
- DigitalOcean App Platform (Budget-friendly, fixed pricing)
- VPS with Docker Compose (Full control, requires DevOps knowledge)



Render Deployment (Recommended)

Prerequisites

- GitHub account with your trader_nightmare repository
- Render account (free tier available)
- Environment variables configured

Step 1: Prepare Repository

```
# Ensure your repository is pushed to GitHub
git remote add origin https://github.com/yourusername/trader_nightmare.git
git push -u origin main
```

Step 2: Deploy Backend on Render

1. Create Web Service:

- Go to [Render Dashboard](https://dashboard.render.com) (https://dashboard.render.com)
- Click "New" → "Web Service"
- Connect your GitHub repository
- Select `trader_nightmare` repository

2. Configure Backend Service:

```
yaml
Name: trader-nightmare-backend
Environment: Docker
Dockerfile Path: ./backend/Dockerfile
Region: Choose closest to your users
Branch: main
```

3. Environment Variables:

```
```bash
SECRET_KEY=your-super-secret-key-change-in-production
DATABASE_URL=sqlite:///app.db
DEBUG=false
LOG_LEVEL=INFO
```

```

VIABILITY_THRESHOLD=70.0
MAX_POSITION_SIZE=1000.0
SLIPPAGE_TOLERANCE=0.02
STOP_LOSS_PERCENTAGE=0.15
TAKE_PROFIT_PERCENTAGE=0.25
MAX_CONCURRENT_POSITIONS=5
CORS_ORIGINS=https://your-frontend-url.onrender.com

API Keys (Add your actual keys)
ALCHEMY_API_KEY=your-alchemy-api-key
QUICKNODE_API_KEY=your-quicknode-api-key
GOPLUS_API_KEY=your-goplus-api-key
```

```

1. **Advanced Settings:**

```

yaml
  Auto-Deploy: Yes
  Health Check Path: /health

```

Step 3: Deploy Frontend on Render

1. **Create Static Site:**

- Click "New" → "Static Site"
- Connect same repository
- Configure build settings:

```

yaml
  Name: trader-nightmare-frontend
  Build Command: cd frontend && npm ci && npm run build
  Publish Directory: frontend/dist

```

1. **Environment Variables:**

```

bash
  VITE_API_URL=https://your-backend-url.onrender.com

```

Step 4: Update CORS Settings

After frontend deployment, update backend environment variables:

```

CORS_ORIGINS=https://your-frontend-url.onrender.com,http://localhost:3000

```

Step 5: Verify Deployment

1. **Backend Health Check:** `https://your-backend-url.onrender.com/health`
2. **Frontend Access:** `https://your-frontend-url.onrender.com`
3. **API Test:** `https://your-backend-url.onrender.com/api/trading/status`

Railway Deployment (Alternative)

Quick Deploy

```
# Install Railway CLI
npm install -g @railway/cli

# Login to Railway
railway login

# Deploy backend
cd backend
railway up

# Deploy frontend
cd ../frontend
railway up
```

Environment Configuration

Set the same environment variables as Render through Railway dashboard.

DigitalOcean App Platform

Using App Spec

Create `app.yaml` :

```
name: trader-nightmare
services:
- name: backend
  source_dir: backend
  dockerfile_path: backend/Dockerfile
  instance_count: 1
  instance_size_slug: basic-xxs
  http_port: 5000
  environment_slug: docker
  envs:
  - key: SECRET_KEY
    value: your-secret-key
  # Add other environment variables

- name: frontend
  source_dir: frontend
  dockerfile_path: frontend/Dockerfile
  instance_count: 1
  instance_size_slug: basic-xxs
  http_port: 80
  environment_slug: docker
```

Deploy:

```
doctl apps create --spec app.yaml
```

VPS Deployment (Advanced)

Prerequisites

- VPS with Docker and Docker Compose installed
- Domain name (optional)
- SSL certificate setup

Deployment Steps

```
# Clone repository
git clone https://github.com/yourusername/trader_nightmare.git
cd trader_nightmare

# Setup environment
cp .env.example .env
nano .env # Edit with your configuration

# Deploy with Docker Compose
docker-compose up -d --build

# Setup reverse proxy (nginx)
sudo apt install nginx
# Configure nginx for SSL and domain routing
```

Nginx Configuration

```
server {
    listen 80;
    server_name your-domain.com;
    return 301 https://$server_name$request_uri;
}

server {
    listen 443 ssl;
    server_name your-domain.com;

    ssl_certificate /path/to/cert.pem;
    ssl_certificate_key /path/to/key.pem;

    location / {
        proxy_pass http://localhost:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    location /api {
        proxy_pass http://localhost:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```



Cost Comparison

| Platform | Small Setup | Medium Setup | Large Setup | Free Tier |
|--------------|--------------|---------------|----------------|------------------|
| Render | \$25/month | \$175/month | \$450/month | ✓ Generous |
| Railway | \$30/month | \$160/month | \$320/month | ⚠ \$5 credit |
| DigitalOcean | \$25/month | \$175/month | \$450/month | ✓ 3 static sites |
| VPS | \$5-20/month | \$20-50/month | \$50-200/month | ✗ None |



Environment Variables Reference

Required Variables

```
# Security
SECRET_KEY=your-super-secret-key-change-in-production

# Database
DATABASE_URL=sqlite:///app.db

# Trading Configuration
VIABILITY_THRESHOLD=70.0
MAX_POSITION_SIZE=1000.0
SLIPPAGE_TOLERANCE=0.02
STOP_LOSS_PERCENTAGE=0.15
TAKE_PROFIT_PERCENTAGE=0.25
MAX_CONCURRENT_POSITIONS=5

# API Keys
ALCHEMY_API_KEY=your-alchemy-api-key
QUICKNODE_API_KEY=your-quicknode-api-key
GOPLUS_API_KEY=your-goplus-api-key
```

Optional Variables

```
# Logging
DEBUG=false
LOG_LEVEL=INFO

# Network
CORS_ORIGINS=https://your-frontend-url.com
BACKEND_PORT=5000
FRONTEND_PORT=3000
```



Monitoring & Maintenance

Health Checks

- **Backend:** `/health` endpoint
- **Frontend:** Root path `/`
- **API Status:** `/api/trading/status`

Logging

- Backend logs: Available in platform dashboards
- Application logs: Stored in `/app/logs` (VPS) or platform logging
- Error tracking: Monitor 4xx/5xx responses

Updates

```
# For Git-based deployments (Render, Railway, DO)
git push origin main # Auto-deploys
```

```
# For VPS
cd trader_nightmare
git pull origin main
docker-compose up -d --build
```

Troubleshooting

Common Issues

- CORS Errors:**
 - Verify `CORS_ORIGINS` includes your frontend URL
 - Check protocol (http vs https)
- Database Connection:**
 - Ensure `DATABASE_URL` is correctly set
 - Check file permissions for SQLite
- API Key Issues:**
 - Verify all required API keys are set
 - Check key validity and rate limits
- Build Failures:**
 - Check Dockerfile syntax
 - Verify all dependencies in `requirements.txt/package.json`

Debug Commands

```
# Check container logs
docker-compose logs backend
docker-compose logs frontend

# Test API endpoints
curl https://your-backend-url.com/health
curl https://your-backend-url.com/api/trading/status

# Verify environment variables
docker-compose exec backend env | grep SECRET_KEY
```

Support

- **Documentation:** [Full Documentation](https://docs.trader-nightmare.com) (https://docs.trader-nightmare.com)
- **Issues:** [GitHub Issues](https://github.com/yourusername/trader_nightmare/issues) (https://github.com/yourusername/trader_nightmare/issues)
- **Community:** [Discord Server](https://discord.gg/trader-nightmare) (https://discord.gg/trader-nightmare)

 **Congratulations!** Your Trader Nightmare bot is now deployed and ready for production trading!

Remember to:

-  Monitor your deployment regularly
-  Keep API keys secure
-  Update dependencies periodically
-  Backup your database
-  Test thoroughly before live trading

 **Trading Disclaimer:** This bot is for educational purposes. Cryptocurrency trading involves substantial risk. Never trade with money you cannot afford to lose.